

Complete 54 DSA Problems with Python Solutions & Explanations

1. Find Largest Element

Explanation: Traverse array and track maximum.

```
def find_largest(arr):  
    return max(arr)
```

2. Reverse a List

Explanation: Reverse using slicing.

```
def reverse_list(arr):  
    return arr[::-1]
```

3. Check Palindrome String

Explanation: Compare string with reverse.

```
def is_palindrome(s):  
    s=s.lower().replace(" ", "")  
    return s==s[::-1]
```

4. Second Largest Element

Explanation: Track largest and second largest.

```
def second_largest(arr):  
    arr=sorted(set(arr))  
    return arr[-2]
```

5. Remove Duplicates

Explanation: Use set while preserving order.

```
def remove_duplicates(arr):  
    return list(dict.fromkeys(arr))
```

6. Find Missing Number

Explanation: Use sum formula.

```
def missing(arr,n):  
    return n*(n+1)//2 - sum(arr)
```

7. Rotate Array

Explanation: Use slicing.

```
def rotate(arr,k):  
    k%=len(arr)  
    return arr[-k:]+arr[:-k]
```

8. Count Vowels

Explanation: Count vowels in string.

```
def count_vowels(s):  
    return sum(1 for c in s if c.lower() in 'aeiou')
```

9. Check Anagrams

Explanation: Sort and compare.

```
def anagram(a,b):  
    return sorted(a)==sorted(b)
```

10. Intersection of Arrays

Explanation: Use set intersection.

```
def intersection(a,b):  
    return list(set(a)&set(b))
```

11. Binary Search

Explanation: Use divide and conquer.

```
def binary_search(arr,target):  
    l,r=0, len(arr)-1  
    while l<=r:  
        m=(l+r)//2  
        if arr[m]==target:  
            return m  
        elif arr[m]<target:  
            l=m+1  
        else:  
            r=m-1  
    return -1
```

12. Bubble Sort

Explanation: Swap adjacent elements repeatedly.

```
def bubble_sort(arr):  
    n=len(arr)  
    for i in range(n):  
        for j in range(n-i-1):  
            if arr[j]>arr[j+1]:  
                arr[j],arr[j+1]=arr[j+1],arr[j]  
    return arr
```

13. Selection Sort

Explanation: Select minimum each pass.

```
def selection_sort(arr):
    for i in range(len(arr)):
        m=i
        for j in range(i+1, len(arr)):
            if arr[j]<arr[m]:
                m=j
        arr[i], arr[m]=arr[m], arr[i]
    return arr
```

14. Insertion Sort

Explanation: Insert element in sorted part.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key=arr[i]
        j=i-1
        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]
            j-=1
        arr[j+1]=key
    return arr
```

15. Find First and Last Position

Explanation: Linear scan for occurrences.

```
def first_last(arr, target):
    return arr.index(target), len(arr)-1-arr[::-1].index(target)
```

16. Check Prime Number

Explanation: Check divisibility till sqrt(n).

```
def is_prime(n):
    if n<2: return False
    for i in range(2, int(n**0.5)+1):
        if n%i==0: return False
    return True
```

17. Factorial

Explanation: Multiply recursively.

```
def factorial(n):
    return 1 if n<=1 else n*factorial(n-1)
```

18. Fibonacci Number

Explanation: Iterative fibonacci.

```
def fibonacci(n):
    a, b=0, 1
    for _ in range(n):
        a, b=b, a+b
    return a
```

19. GCD

Explanation: Use Euclidean algorithm.

```
def gcd(a,b):  
    while b:  
        a,b=b,a%b  
    return a
```

20. LCM

Explanation: Use gcd formula.

```
def lcm(a,b):  
    import math  
    return abs(a*b)//math.gcd(a,b)
```

21. Armstrong Number

Explanation: Check power sum of digits.

```
def armstrong(n):  
    p=len(str(n))  
    return sum(int(d)**p for d in str(n))==n
```

22. Perfect Number

Explanation: Sum divisors.

```
def perfect(n):  
    return sum(i for i in range(1,n) if n%i==0)==n
```

23. Sum of Digits

Explanation: Add digits.

```
def sum_digits(n):  
    return sum(map(int,str(n)))
```

24. Reverse Number

Explanation: Reverse digits.

```
def reverse_num(n):  
    return int(str(n)[::-1])
```

25. Palindrome Number

Explanation: Compare with reverse.

```
def palindrome_num(n):  
    return str(n)==str(n)[::-1]
```

26. Right Triangle Pattern

Explanation: Print star triangle.

```
for i in range(1,5):
    print("* "*i)
```

27. Pyramid Pattern

Explanation: Centered star pyramid.

```
for i in range(1,5):
    print(" "*(4-i)+"*"(2*i-1))
```

28. Number Pyramid

Explanation: Print incremental numbers.

```
for i in range(1,5):
    print(*range(1,i+1))
```

29. Diamond Pattern

Explanation: Upper and lower pyramids.

```
# combine two pyramids
```

30. Hollow Square

Explanation: Print borders only.

```
# nested loop pattern
```

31. Power Function

Explanation: Recursive power.

```
def power(a,b):
    return a**b
```

32. Sum of Natural Numbers

Explanation: Formula $n(n+1)/2$.

```
def sum_n(n):
    return n*(n+1)//2
```

33. Print Numbers Recursively

Explanation: Recursive printing.

```
def print_nums(n):
```

```
if n:
    print_nums(n-1)
    print(n)
```

34. Tower of Hanoi

Explanation: Recursive disk movement.

```
def hanoi(n,s,d,a):
    if n==1:
        print(s,d)
        return
    hanoi(n-1,s,a,d)
    print(s,d)
    hanoi(n-1,a,d,s)
```

35. Count Digits

Explanation: Use string length.

```
def count_digits(n):
    return len(str(n))
```

36. Convert to Uppercase

Explanation: Use upper().

```
def upper(s):
    return s.upper()
```

37. Count Words

Explanation: Split by spaces.

```
def count_words(s):
    return len(s.split())
```

38. Remove Spaces

Explanation: Replace spaces.

```
def remove_spaces(s):
    return s.replace(" ","")
```

39. Count Character Occurrences

Explanation: Use count().

```
def occurrences(s,ch):
    return s.count(ch)
```

40. String to Integer

Explanation: Convert manually.

```
def to_int(s):  
    return int(s)
```

41. Toggle Case

Explanation: Swap upper/lower.

```
def toggle(s):  
    return s.swapcase()
```

42. String Rotation

Explanation: Check substring.

```
def rotation(a,b):  
    return len(a)==len(b) and b in a+a
```

43. Sum of Natural Numbers Again

Explanation: Formula approach.

```
def natural_sum(n):  
    return n*(n+1)//2
```

44. Prime Numbers in Range

Explanation: Check each number.

```
def primes(a,b):  
    return [n for n in range(a,b+1) if all(n%i for i in range(2,int(n**0.5)+1)) and n>1]
```

45. Automorphic Number

Explanation: Square ends with number.

```
def automorphic(n):  
    return str(n*n).endswith(str(n))
```

46. Neon Number

Explanation: Digit sum of square equals number.

```
def neon(n):  
    return sum(map(int,str(n*n)))==n
```

47. Abundant Number

Explanation: Divisor sum greater than number.

```
def abundant(n):  
    return sum(i for i in range(1,n) if n%i==0)>n
```

48. Happy Number

Explanation: Detect cycle with set.

```
def happy(n):
    seen=set()
    while n!=1 and n not in seen:
        seen.add(n)
        n=sum(int(d)**2 for d in str(n))
    return n==1
```

49. Equilibrium Index

Explanation: Left sum equals right sum.

```
def equilibrium(arr):
    total=sum(arr)
    left=0
    for i,x in enumerate(arr):
        total-=x
        if left==total: return i
        left+=x
    return -1
```

50. Kadane Algorithm

Explanation: Maximum subarray sum.

```
def kadane(arr):
    cur=best=arr[0]
    for x in arr[1:]:
        cur=max(x,cur+x)
        best=max(best,cur)
    return best
```

51. Move Zeros to End

Explanation: Shift non-zero elements.

```
def move_zeros(arr):
    nz=[x for x in arr if x!=0]
    return nz+[0]*(len(arr)-len(nz))
```

52. Alternate Positive Negative

Explanation: Merge positive and negative.

```
# separate and merge alternately
```

53. Leaders in Array

Explanation: Traverse from right.

```
def leaders(arr):
    m=arr[-1]
    res=[m]
```



```
for i in arr[-2::-1]:
    if i>m:
        res.append(i)
        m=i
return res[::-1]
```

54. Spy Number

Explanation: Sum equals product of digits.

```
def spy(n):
    s,p=0,1
    for d in str(n):
        s+=int(d)
        p*=int(d)
    return s==p
```